

Assignement-18

7 Explain diff betⁿ static & dynamic memory allocation
 ⇒ ~~Static~~ Static = S Dynamic = D

7. S - No specific function
 ① - specific function for memory allocation & deallocation.

Q1 3 - cell of memory allocatⁿ at compile time
0 - 1 - runtime

3) memory Allocation - at runtime [for both]

7] S - may lead to problem of memory storage/wastage
D - No such problem

5/ 3 - Can't deallocate memory, once its use gets completed
D - We can reallocate ——— 1 ———

67 S - No memory allocⁿ failure at run
D - Can be failure due to 'Shortage of memory'

#7] 3 - Fast way of memory allocation as there is no call at runtime.

① - slowest way of memory allocⁿ as cal^d at runtime.

5) $\Rightarrow$ memory allocated inside memory
[data, stack, CPU, register]

① - memory allocate inside 'Heap'
[In C++ Heap = 'Free Store']

2] What are advantage & disadvantage of dynamic memory allocation over static memory allocation.

→ Advantages:

- 1] Efficient memory Utilization:
Memory allocated as needed at runtime, avoids wastage
- 2] Flexibility: Enable creating data structures like linked lists, trees & graphs with sizes unknown at compile time
- 3] Scalability: You can resize memory (e.g. using realloc) if requirements change
- 4] Dynamic Lifetime: Objects can exist beyond the scope of function that created them.

× Disadvantages:

- 1] Overhead: Involves runtime computations, which may slow down the program
- 2] Manual management: Requires explicit deallocation (free() in C, delete in C++), risking memory leaks.
- 3] Fragmentation: Memory blocks may become fragmented leading to inefficient usage
- 4] Error-prone: Incorrect handling may cause issues like double-free, dangling pointers, or memory corruption.

3] List down functions & its syntaxes which are used in C programming for dynamic memory allocation

- 1] malloc() → void *malloc (datatype size)
- 2] free() → void free (void *ptr)
- 3] calloc() → void *calloc (datatype-no. of elements, datatype size - of - each - element);
- 4] realloc() → void *realloc (void *ptr, datatype new-size);

4) Which functions/operators are used in C++ programming for dynamic memory allocation & deallocations?

→ 1) Allocation

new : Allocates memory for a single object.

Syntax : `Type *ptr = new Type;`

new[] : Allocates memory for an array of objects.

Syntax : `Type *ptr = new Type[size];`

2) Deallocation

delete : Frees memory allocated for single object.

Syntax → `delete ptr;`

delete[] : Frees memory allocated for an array of objects.

Syntax → `delete[] ptr;`

5) Diff betⁿ malloc() & calloc()

malloc()

calloc()

Initializes : Does not initialize memory

Initializes memory to zero

Syntax : `void *malloc(size_t size);`

`void *calloc(size_t n, size_t size);`

Arguments : Single argument (size in bytes)

2 arguments (elements

& size per element)

Use case : General-purpose allocation

Useful for initializing arrays

6) Explain prototype of malloc function with example.

Syntax - `void * malloc (datatype size)`

* Code - dynamic1.cpp

`#include <stdlib.h>`

`#include <iostream>`

using namespace std;

```

int main()
{
    int arr[5]; // static memory allocation
    int *ptr1 = (int *) malloc (5 * sizeof (int));
    free (ptr1);
    int *ptr2 = new int [5]; // dynamic memory allocation
    delete [] ptr2; // deallocation
    return 0;
}

```

1 more Code for reference → dynamic1.c

- return value of `malloc()` is 'void *', indicates address of allocated memory
- we have to type cast, after getting address
- `malloc()` accepts only 1 parameter
- i.e. no. of bytes that we want to allocate (size)

7] Why return value of `malloc()`, `calloc()` and `realloc()` is void *?

→ Return value of `malloc()` is 'void *', indicates address of allocated memory

- Generic Pointer - void* is a generic pointer that can be cast to any type (int*, float*, etc.)
- Flexibility - Allows these functions to handle diff data types w/o needing separate implementation
- Manual Casting - User explicitly casts the return value to desired type.

8] What are difference uses of `realloc()`?

-
- 1] Used to 'Resize' the allocated memory size
 - 2] Used to increase or decrease the size of already
 - 3] Others : 1] Allocate fresh memory
2] Deallocate allocated memory

Q3. `int *ptr = (int*) malloc (5 * sizeof (int));`

if increase memory allocated:

`int *ptr = (int*) realloc (ptr, 32);`

2) Decrease memory allocated

`int *ptr = (int*) realloc (ptr, 16);`

9) What will happen if we use 1st parameter of `realloc()` as NULL?

→ If 1st parameter of `realloc()` is NULL, it will work like `malloc()`

Ex: `int *ptr1 = (int*) realloc (NULL, (30));`

10) What will happen if 2nd parameter of `realloc()` is 0?

→ If 2nd parameter of `realloc()` is '0' (zero) then `realloc()` will work like `free()`.

Ex: `int *ptr1 = (int*) realloc (ptr, (0));`